

Managing Multiple Tasks in Highly Redundant Robotic Systems

Presentation based on Siciliano & Slotine (1991)

Robin DIDIER & Lubin CAILLOL

Fundation of robotics

May 2026

Table of Contents

- Motivations
- The task-priority framework
- Limitations & implementation
- Simulation Showcase
- Conclusion & future evolutions

Motivations

Robot redundancy

- Joint space: $q \in \mathbb{R}^n$ (angles/distances).
- Task space (End-effector): $x \in \mathbb{R}^m$ (position/orientation).
- Direct Kinematics: $x = f(q)$.

By differentiating, we obtain velocity mapping:

$$\dot{x} = J\dot{q}$$

where $J = \frac{\partial f}{\partial q}$ is the Jacobian matrix ($m \times n$).

Redundancy: Occurs when $n > m$ (more motors than task constraints).

Standard solution & null-space

Redundant robot $\rightarrow$ infinite solutions to achieve a desired $\dot{x}$. We use the Moore-Penrose pseudo-inverse $J^\#$:

$$\dot{q} = J^\# \dot{x} + (I - J^\# J)z$$

Physical Interpretation:

- $J^\# \dot{x}$: Minimum-norm joint velocity to achieve the task.
- $(I - J^\# J)$: The projector onto the **null-space** of J .
- z : An arbitrary velocity vector. It is filtered so its effect on the end-effector ($\dot{x}$) is strictly zero (self-motion).

Previous approaches: the acceleration problem

Control computed at acceleration level :

$$\ddot{x} = J\ddot{q} + \dot{J}\dot{q}.$$

Solving for $\ddot{q}$:

$$\ddot{q} = J^\#(\ddot{x} - \dot{J}\dot{q}) + (I - J^\#J)z.$$

Unstable !

- No velocity feedback in the null-space.
- Any numerical error in $\ddot{q}$ integrates twice, causing *velocity drift*.

And what about multiple tasks ?

Paradigm shift

What we are going to see...

The Siciliano & Slotine approach

- Core resolution to the **velocity level**.
- Computing $\dot{q}$ first $\rightarrow$ internal velocities bounded.
- Deriving accelerations analytically.
- Managing multiple tasks.

The task-priority framework

Task hierarchy & augmented Jacobian

Assume k simultaneous tasks. Task 1 is highest priority, Task k is lowest.

For each task i :

- $\dot{x}^i$: Desired velocity for task i .
- J^i : Jacobian associated with task i .

We define the **Augmented Jacobian** J_A^{i-1} as the stack of all higher-priority Jacobians:

$$J_A^{i-1} = \begin{pmatrix} J^1 \\ J^2 \\ \vdots \\ J^{i-1} \end{pmatrix}$$

The core theorem

Let $\dot{q}^{i-1}$ be the solution satisfying the first $i - 1$ tasks. The update rule to satisfy task i is:

$$\begin{aligned}\dot{q}^i &= \dot{q}^{i-1} + \bar{J}^i \# (\dot{x}^i - J^i \dot{q}^{i-1}) \\ \dot{q}^1 &= J^1 \# \dot{x}^1\end{aligned}$$

Where:

- $\bar{J}^i = J^i P_A^{i-1}$: The projected Jacobian.
- $P_A^{i-1} = I - (J_A^{i-1})^\# J_A^{i-1}$: The projector onto the null-space of all higher-priority tasks.

The principle of non-interference

Intuition :

1. **Baseline:** $\dot{q}^{i-1}$ executes tasks $1 \dots i - 1$. We must not break this.
2. **Residual Effort:** $(\dot{x}^i - J^i \dot{q}^{i-1})$ is what is missing to achieve task i .
3. **Constrained Modification:** We don't use J^i , but the projected $\bar{J}^i$.

If J^i is what task i *wants* to do, $\bar{J}^i$ is what task i *is allowed* to do.

The corrective motion lives in the null-space of superior tasks.

Limitations & implementation

Singularities & damped least-squares

The recursive framework faces two types of singularities:

- **Task Singularity:** J^i loses rank (physical limit).
- **Algorithmic Singularity:** $\bar{J}^i$ loses rank (Task i is physically impossible without violating higher-priority tasks).

Near singularities, the pure pseudo-inverse generates infinite velocities.

→ **Damped least-squares.** It safely caps joint velocities at the cost of a slight, controlled tracking error.

Simulation Showcase

Live simulation: 7-DOF planar robot

Implemented Hierarchy:

1. Obstacle avoidance (max priority)
2. End-effector orientation
3. End-effector tracking
4. Posture optimization

Let's see this...

Conclusion & future evolutions

From pseudo-inverses to optimization

Fundamental limitation:

- The framework is built for strict **equality constraints** ($\dot{x} = J\dot{q}$).
- Tasks like obstacle avoidance are natively **inequalities** ($J\dot{q} \leq b$).

Modern Evolution:

- Shift towards **Hierarchical Quadratic Programming** (Kanoun et al., 2011).
- Cascade of optimization problems, handling inequalities.

Thank you for your attention.

Any questions?

Appendix A: Proof of the null-space projector

Goal: Prove that $P = I - J^\# J$ is a projector ($P^2 = P$) and $Pz \in \text{Ker}(J)$.

1. Idempotency property ($P^2 = P$):

- $P^2 = (I - J^\# J)(I - J^\# J) = I - 2J^\# J + J^\# J J^\# J$
- Using the property $J J^\# J = J$, we get:
- $P^2 = I - 2J^\# J + J^\# J = I - J^\# J = P \quad \checkmark$

2. Kernel property ($JP = 0$):

- $J(I - J^\# J) = J - J J^\# J = J - J = 0 \quad \checkmark$

Any velocity z multiplied by P will result in zero end-effector motion.

Appendix B: 3-DOF Case Study

Consider a planar robot with 3 joints $q = [q_1, q_2, q_3]^T$.

Defining the tasks:

1. **Task 1 (Priority):** Lock the first joint ($\dot{q}_1 = 0$).
 - ▶ Task space: $x^1 = q_1$
 - ▶ Jacobian (Selection matrix): $J^1 = \begin{pmatrix} 1 & 0 & 0 \end{pmatrix}$
2. **Task 2 (Secondary):** Move the end-effector at velocity $\dot{x}^2 = [\dot{p}_x, \dot{p}_y]^T$.
 - ▶ Geometric Jacobian: $J^2 = \begin{pmatrix} J_{11}^2 & J_{12}^2 & J_{13}^2 \\ J_{21}^2 & J_{22}^2 & J_{23}^2 \end{pmatrix}$

Appendix B: 3-DOF Case Study

1. Computing the first projector P_A^1 :

$$P_A^1 = I - J^{1\#} J^1 = I - \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} (1 \ 0 \ 0) = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

2. Projecting the secondary task:

$$\bar{J}^2 = J^2 P_A^1 = \begin{pmatrix} J_{11}^2 & J_{12}^2 & J_{13}^2 \\ J_{21}^2 & J_{22}^2 & J_{23}^2 \end{pmatrix} \begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & J_{12}^2 & J_{13}^2 \\ 0 & J_{22}^2 & J_{23}^2 \end{pmatrix}$$

Appendix B: 3-DOF Case Study

The first column of $\bar{J}^2$ is zero. The pseudo-inverse $\bar{J}^{2\#}$ will have a zero first row.
Using

$$\dot{q}^2 = \dot{q}^1 + \bar{J}^{2\#}(\dot{x}^2 - J^2 \dot{q}^1)$$

with $\dot{q}_1 = 0$, we have

$$\dot{q}^2 = \bar{J}^{2\#} \dot{x}^2.$$

Result:

Joint 1 is mathematically "locked" for the tracking task.

Appendix C: Damped Least-Squares (DLS)

The problem: Near singularities, $\det(JJ^T) \rightarrow 0$, leading to $\|J^\#\| \rightarrow \infty$.

The solution: Replace $J^\#$ with the damped version J_λ^* :

$$J_\lambda^* = J^T(JJ^T + \lambda^2 I)^{-1}$$

Trade-off:

- λ (Damping factor): Small constant (e.g., $0.01 \dots 0.1$).
- **High manipulability:** $J_\lambda^* \approx J^\#$ (High precision).
- **Near singularity:** Velocities are capped at $\|\dot{q}\| \approx \frac{1}{2\lambda} \|\dot{x}\|$.

Appendix D: Detailed Comparison - S&S vs HQP

Feature	Siciliano & Slotine (1991)	Modern HQP
Mathematical Core	Pseudo-inverse / Projection	Numerical Optimization (QP)
Constraints	Equalities only ($\dot{x} = J\dot{q}$)	Equalities AND Inequalities
Transitions	Sudden "On/Off" behavior	Smooth and continuous
Computation	Analytical (Extremely fast)	Iterative (Requires solver)

Hierarchical quadratic programming is now the gold standard for humanoid robots.